• System design

What does the retry have permission for?

An approval diagram needs room for the returning arrow.

What still authorizes it?

A proposed design review.

• System design

The reply can be lost after the action succeeds.

A timeout leaves the caller uncertain.
Repeating a request may repeat its effect.

No reply
≠ proof of no effect

Retry semantics: RFC 9110 §9.2.2.

• System design

Ask whether it is the same intended action.

A repeated request and a new instruction can look similar. I would make the intent explicit.

01 Which action was approved?

02 What limits applied?

03 Do they still cover this request?

Approval scope is a separate check.

• System design

Give the request an identity that survives.

Use the service's idempotency contract to identify a retry. Check how it treats changed parameters and expired records.

Same identifier?
Same intended effect?

Do not infer intent from matching text alone.

System design

The record needs to survive the failure too.

Writing “done” locally after the effect leaves a crash window. I would check how the effect and its record stay consistent.

01 Inspect the service’s guarantees.

02 Reconcile uncertain outcomes.

03 Keep unresolved attempts visible.

A durable log alone does not ensure one effect.

System design

Then lose the reply on purpose.

In a controlled test, let the action take effect, suppress its response, then exercise the retry path.

01 Count the actual effects.

02 Trace the approval used.

03 Inspect the returned result.

Suggested test; no deployment result claimed.

• System design

Change the intent and test the boundary.

I would also try changed parameters, an expired approval and a late retry. The intended response should be explicit.

A familiar request can need a new decision.

Reauthorization depends on the actual scope.

• System design

Draw the returning arrow before trusting the diagram.

I want to see how the same action is recognised, how uncertain outcomes are resolved, and where approval stops applying.

Reading: Malcolm Featonby, Making retries safe with idempotent APIs.
RFC 9110 §9.2.2.
Approval review: my proposed design questions.